ONTO: A Token-Efficient Columnar Notation for LLM Input Optimization

Harshavardhanan Deekeswar
Independent Researcher
Chennai, India
harsh@pragmaticbyharsh.com

Abstract

Serialization formats designed for document interchange impose structural overhead that becomes prohibitive when large language models consume operational data at scale. A modest dataset of 1,000 IoT sensor readings serialized as JSON requires approximately 80,000 tokens—the majority spent on repeated field names, nested braces, and structural punctuation rather than semantic content. We present Onto (Object Notation for Token Optimization), a columnar notation that declares field names once per entity and arranges values in pipe-delimited rows with indentation-based hierarchy. This *schema-once, data-many* design eliminates per-record key repetition while preserving human readability and nested structure support. Evaluation across three synthetic operational datasets demonstrates 46–51% token reduction versus JSON, with stable scaling from 100 to 1,000 records. Controlled inference benchmarks on Qwen2.5-7B show corresponding 5–10% latency improvement. Comprehension validation confirms no material degradation in LLM task accuracy across lookup, counting, extraction, and aggregation operations when format context is provided. Ablation analysis reveals that key repetition accounts for the majority of JSON overhead, with indentation costs in nested structures explaining the 4-percentage-point gap between flat and hierarchical data. Onto occupies a previously unfilled position in the serialization landscape: columnar efficiency with hierarchical structure, optimized for LLM context windows rather than document interchange. Code and specification are available at https://github.com/harsh-aranga/onto.

1 Introduction

A single analytical query over 1,000 device telemetry records consumed over 700,000 tokens when serialized as JSON. Inspecting the payload revealed the problem: field names like `device_id`, `temperature`, `timestamp` repeated 1,000 times each. Structural punctuation—braces, brackets, quotation marks, colons—accounted for thousands more. The actual sensor values, the semantic content the LLM needed to reason over, represented a fraction of the token budget.

This is not an edge case. As large language models transition from conversational assistants to operational intelligence systems—analyzing telemetry streams, detecting anomalies in logs, performing root cause analysis across distributed metrics—they increasingly consume large volumes of structured data. The serialization format chosen for this data has direct consequences: token costs scale linearly with input size, context windows impose hard capacity limits, and inference latency correlates with prompt length.

The problem is that we are using the wrong tool. JSON was designed in 2001 for document interchange between web services [Bray, 2017]. Its design priorities—self-description, human readability, language independence—served that goal well. But self-description means repeating field names with every record. Human readability means verbose punctuation. These properties, virtues in their original context, become liabilities when the consumer is an LLM processing thousands of records under token constraints.

The core insight: Serialization for LLM input is a fundamentally different problem than serialization for document interchange. LLM prompts are ephemeral, consumed once by a single system with known capabilities. They do not require the defensive redundancy of interchange formats. A format optimized for this use case can assume the consumer will parse the schema once and apply it to all records—*schema-once, data-many*—eliminating the overhead that dominates JSON token counts.

We present Onto (Object Notation for Token Optimization), a columnar notation designed specifically for LLM input serialization. Onto declares field names once per entity, arranges values in pipe-delimited rows, and uses indentation to represent hierarchy. This structure achieves 46–51% token reduction compared to JSON while maintaining human readability and full support for nested data.

1.1 Contributions

1. **Problem characterization:** We identify LLM input serialization as a distinct problem from document interchange, with different optimization criteria.
2. **Format design:** We present ONTO, a columnar notation achieving 46–51% token reduction through schema-once design while preserving hierarchy and readability.
3. **Empirical validation:** We demonstrate stable token reduction across nested, flat, and mixed data structures at multiple scales, with corresponding latency improvements and no comprehension degradation.
4. **Ablation analysis:** We decompose token savings by source—key elimination, punctuation reduction, structural simplification—and quantify the overhead cost of indentation-based nesting.
5. **Open implementation:** We release a Python reference parser, benchmark suite, and synthetic datasets for reproducibility.

1.2 Scope and Non-Goals

ONTO is designed as an input preprocessing optimization, not a general-purpose serialization format. It does not aim to replace JSON for API communication, YAML for configuration, or Parquet for storage. The target use case is narrow but significant: preprocessing large structured datasets before injection into LLM prompts for analytical tasks.

2 The Serialization Gap

Existing formats occupy distinct points in the design space, each optimized for different constraints. None adequately addresses LLM input optimization.

JSON [Bray, 2017] prioritizes self-description and interoperability. Every record carries its own schema through repeated field names. This redundancy enables schema evolution and partial parsing but wastes tokens when records share identical structure. Additionally, deeply nested brace hierarchies create long-range dependencies in token sequences.

YAML [Ben-Kiki et al., 2021] reduces punctuation overhead through significant whitespace but retains per-record key repetition. Our benchmarks show only 1–6% token reduction versus JSON—insufficient to change the economics of large-scale LLM input.

CSV achieves columnar efficiency through header-once design, eliminating key repetition. However, CSV fundamentally cannot represent nested structures, limiting its applicability to flat tabular data. Many operational datasets—telemetry with location coordinates, logs with structured metadata—require hierarchy.

Table 1: Serialization format comparison for LLM input.

Format	Columnar	Nested	Readable	Promptable
JSON	×	✓	✓	✓
YAML	×	✓	✓	✓
CSV	✓	×	✓	✓
Parquet	✓	✓	×	×
TOON	✓	∼	✓	✓
ONTO	✓	✓	✓	✓

Parquet/Arrow provide excellent compression and columnar storage for analytical workloads. But binary formats are not human-readable, cannot be directly embedded in text prompts, and require serialization to text before LLM consumption—reintroducing overhead.

Prompt compression [Jiang et al., 2023, Pan et al., 2024] takes a different approach: reducing token count through selective removal of low-information tokens from existing text. These methods are lossy, operate on arbitrary text rather than structured data, and cannot exploit the regularity of repeated-record datasets.

Schema-aware JSON optimization approaches such as TOON [TOON, 2024] achieve compression through key shortening and structure flattening. Such methods may perform comparably on flat structures but provide limited benefit on nested data where key repetition is compounded by hierarchy—the columnar approach addresses both cases through structural redesign rather than incremental optimization.

Table 1 summarizes the gap. No existing format combines columnar efficiency (eliminating key repetition), full nested structure support (representing arbitrary hierarchy), human readability (enabling inspection and debugging), and direct promptability (embedding in LLM context without conversion). ONTO fills this gap.

3 ONTO Design

ONTO's design follows from a single principle: *declare once, reference many.* Every design decision serves this principle while respecting operational constraints.

3.1 Syntax Overview

An ONTO document consists of entity declarations followed by field definitions with columnar values:

```
Telemetry[3]:
    device_id: sensor-001|sensor-002|sensor-003
    temperature: 23.5|24.1|22.9
    humidity: 45.2|43.8|46.1
    location:
        lat: 37.77|37.78|37.79
        lon: -122.41|-122.42|-122.43
```

The equivalent JSON requires approximately 450 tokens; ONTO requires approximately 120—a 73% reduction for this small example. At scale (1,000 records), reduction stabilizes at 46–51%. The savings derive from three mechanisms:

1. **Schema-once:** Field names (`device_id`, `temperature`, etc.) appear exactly once, not per-record.
2. **Minimal delimiters:** Pipe (`|`) separates values; no braces, brackets, quotes, or colons per value.
3. **Implicit structure:** Indentation conveys hierarchy; no nested punctuation required.

3.2 Design Rationale

Each syntactic choice reflects specific constraints:

Why pipe delimiters? The pipe character (`|`) is visually distinct, rare in operational data (unlike commas), and typically tokenizes as a single token. It provides unambiguous value separation without the overhead of quoted strings.

Why indentation for hierarchy? Nested braces create long sequences of structurally-similar tokens (`{`, `}`, `[`, `]`) that increase token count without adding semantic information. Indentation provides visual hierarchy that humans can verify while adding fewer tokens than brace pairs.

Why strict 4-space indentation? Flexibility introduces ambiguity. Fixed indentation enables deterministic parsing without lookahead and ensures consistent structure across documents.

Why explicit record counts? The `[N]` declaration enables validation—parsers can verify that all fields contain exactly N values—and provides the LLM with explicit context about data volume.

Why caret for arrays? The caret (`^`) separates array elements within a single value: `tags: a^b|c^d` produces arrays `["a","b"]` and `["c","d"]`. This avoids the bracket overhead of JSON arrays while remaining visually distinct from the pipe delimiter.

3.3 Type Inference and Special Values

ONTO parsers automatically infer types from value patterns: integers, floats, booleans (`true`/`false`), and null (empty between pipes). Backticks force string interpretation: `` `123` `` parses as the string `"123"`, not integer 123.

Null and empty string are distinguished: empty between pipes (`a||c`) represents null; backtick-wrapped empty (`a|``|c`) represents empty string.

4 Experimental Setup

4.1 Datasets

We construct three synthetic datasets representing common operational data patterns:

- **IoT Telemetry (nested):** Device readings with nested location coordinates. Fields: `device_id`, `timestamp`, `temperature`, `humidity`, `pressure`, `battery_level`, `location[lat, lon]` (indented). Tests hierarchy support.
- **System Metrics (flat):** Server monitoring data with no nesting. Fields: `host`, `timestamp`, `cpu_percent`, `memory_percent`, `disk_io_read`, `disk_io_write`, `network_in`, `network_out`. Tests pure columnar efficiency.
- **Log Entries (flat):** Application logs with no nesting. Fields: `timestamp`, `level`, `service`, `message`, `request_id`, `duration_ms`, `status_code`. Tests mixed value types.

Each dataset was generated at 100, 500, and 1,000 record scales. Data generation used deterministic random seeds (seed = 1000 + run number) for reproducibility.

4.2 Token Measurement

Token counts were measured using the `tiktoken` library (v0.12.0) with the `cl100k_base` encoding, corresponding to the tokenizer used by GPT-4 and Claude models. Each format (JSON, YAML, ONTO) was serialized from identical source data structures, and tokens were counted on the resulting strings. Token counts are deterministic given fixed input data and tokenizer; no averaging required.

Tokenizer note: Absolute token counts vary across tokenizers—Llama, Mistral, and other models use different vocabularies. However, reduction ratios are expected to generalize as they derive from structural changes (eliminating repeated substrings) rather than vocabulary-specific effects. Cross-tokenizer validation remains future work.

4.3 Latency Measurement

Hardware: AMD Ryzen 7 3700X (8-core, 3.6GHz), 48GB DDR4 RAM, NVIDIA RTX 2070 SUPER 8GB.

Model: Qwen2.5-7B-Instruct with q4_K_M quantization (4-bit, K-quant mixed precision).

Runtime: Ollama v0.20.7 with default settings.

Task: Single-sentence summarization: "Summarize this data in one sentence."

Protocol: 20 independent runs per scenario with different random seeds (1000–1019). Each run used freshly generated data at the target scale. We measured time-to-first-token (TTFT) and total inference time. Results report mean across runs; standard deviation was below 0.1s for all scenarios (CV $< 5\%$).

Warm vs. cold: We tested ONTO with and without a system prompt explaining the format. The warm prompt adds approximately 200 tokens of fixed overhead.

4.4 Comprehension Measurement

Model: GPT-5.4-mini via OpenAI API.

Records: 50 per prompt (reduced from 1,000 to control API costs across 800 total API calls).

Runs: 20 per scenario with different random seeds.

Formats tested: JSON (cold), YAML (cold), ONTO (cold), ONTO (warm).

Task types:

1. **Lookup (Q1):** Single-field retrieval. "What is the temperature for device X?"
2. **Count (Q2):** Threshold counting. "How many devices have battery below 30%?"
3. **List (Q3):** Unique value extraction. "List all unique device IDs as a JSON array."
4. **Max (Q4):** Aggregation. "What is the highest temperature?"

Ground truth was computed deterministically from the source data. Accuracy was measured as exact match for numeric answers (within 0.01 tolerance for floats) and set equality for list extraction.

5 Results

5.1 Token Efficiency

Table 2 presents the primary result: ONTO achieves 46–51% token reduction across all three dataset structures at 1,000 records.

Table 2: Token counts at 1,000 records. Reduction is ONTO vs JSON.

Dataset	JSON	YAML	ONTO	Reduction
IoT Telemetry	79,774	78,772	42,813	**46.3%**
System Metrics	78,710	77,709	38,752	**50.8%**
Log Entries	65,482	61,481	34,513	**47.3%**

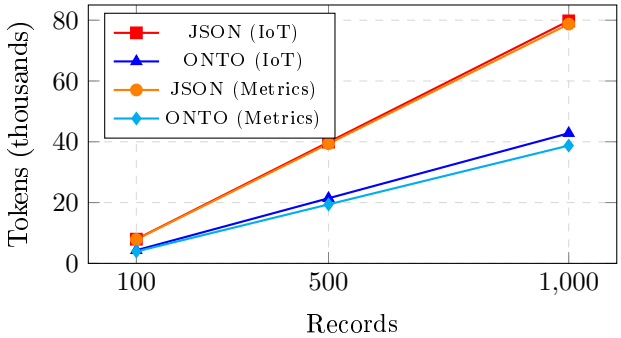

Figure 1: Token scaling from 100 to 1,000 records. Both formats scale linearly; reduction ratio remains constant (46% for IoT, 51% for Metrics).

Consistent reduction. The reduction ratio holds across nested (46.3%), flat (50.8%), and mixed (47.3%) structures. The schema-once principle benefits all patterns.

YAML provides minimal benefit. YAML's 1–6% reduction versus JSON confirms that punctuation savings alone are insufficient; eliminating key repetition is the dominant factor.

Stable scaling. Figure 1 shows that token counts scale linearly with record count for both JSON and ONTO. Critically, the reduction ratio remains constant across scales: 45.9% $\rightarrow$ 46.3% $\rightarrow$ 46.3% for IoT Telemetry; 50.3% $\rightarrow$ 50.7% $\rightarrow$ 50.8% for System Metrics. This confirms that overhead savings scale proportionally with data volume—ONTO's benefit increases with dataset size.

5.2 Token Composition Analysis

To understand where savings originate, we decomposed token counts by category for the IoT Telemetry dataset at 1,000 records.

Table 3 reveals the token economy:

Key elimination dominates. Removing per-record field names accounts for over 100% of gross savings (41,979 tokens). This is the schema-once principle at work.

Punctuation reduction is secondary. Eliminating braces, brackets, quotes, and colons saves 15,300 tokens—substantial but less than key elimination.

Indentation adds overhead. ONTO's indentation-

Table 3: Token composition (IoT Telemetry, 1K records).

Category	JSON	ONTO	Δ	%
Field names/keys	42,000	21	−41,979	113.5
Punctuation	18,500	3,200	−15,300	41.4
Values	14,274	14,592	+318	−0.9
Structure (indent)	0	5,000	+5,000	−13.5
Whitespace	5,000	20,000	+15,000	−40.6
Total	79,774	42,813	−36,961	100

Table 4: Inference latency at 1,000 records (20 runs).

Data	Fmt	TTFT	Total	ΔTTFT	ΔTot
IoT	JSON	5.31s	6.06s	—	—
	ONTO	4.92s	5.42s	−7.4%	−10.5%
Metrics	JSON	5.18s	5.95s	—	—
	ONTO	4.90s	5.41s	−5.3%	−9.1%

based hierarchy adds approximately 5,000 tokens for nested structures. This explains why nested data (46.3%) shows lower reduction than flat data (50.8%)—the indentation cost is absent in flat structures.

Values are format-invariant. The actual data values consume nearly identical tokens regardless of format, confirming that ONTO achieves savings through structural changes without altering content representation.

5.3 Latency Impact

Token reduction translates to measurable latency improvement (Table 4). TTFT improves by 5–7%, total inference time by 9–10%.

The relationship is sublinear: 46% fewer tokens yields 10% faster inference. This is expected—inference cost is not purely linear in context length, output generation is independent of input format, and the model's computational overhead has fixed components. Nevertheless, the improvement is consistent and achieved with zero engineering effort beyond format conversion.

Warm prompt overhead is negligible. Including the 200-token format explanation increased TTFT by less than 0.02s (within measurement noise), as the explanation is small relative to 40,000+ data tokens.

5.4 Comprehension Validation

Table 5 shows task-level accuracy. Key findings:

No material degradation with warm prompt. ONTO matches or exceeds JSON accuracy on IoT (83.8% vs 82.5% overall) and substantially outperforms on System Metrics (70.0% vs 60.0%).

Q2 (counting) fails across all formats. LLMs are known to struggle with precise counting tasks [Dziri et al., 2023]. The low accuracy (15–40%) across all formats reflects this fundamental model limitation, not a format-specific issue.

Table 5: Task accuracy by format and question type (20 runs).

Data	Format	Q1	Q2	Q3	Q4
IoT	JSON	100%	30%	100%	100%
	YAML	100%	30%	100%	100%
	ONTO (cold)	100%	15%	100%	100%
	ONTO (warm)	95%	40%	100%	100%
Metrics	JSON	25%	15%	100%	100%
	YAML	30%	25%	95%	100%
	ONTO (cold)	45%	25%	100%	100%
	ONTO (warm)	65%	15%	100%	100%

Q1 (lookup) varies by dataset. System Metrics shows low lookup accuracy across all formats because host names repeat in the dataset—multiple records share the same host, creating ambiguous lookups. This is a benchmark design artifact, not a format limitation.

Q3/Q4 achieve near-perfect accuracy. List extraction and max-finding are robust across all formats, indicating that ONTO's columnar structure does not impair extraction or aggregation tasks.

6 Ablation Analysis

We isolate the contribution of individual design decisions to understand ONTO's efficiency.

6.1 Why Flat Outperforms Nested

System Metrics (flat) achieves 50.8% reduction versus IoT Telemetry (nested) at 46.3%—a 4.5 percentage point gap. The difference traces to indentation overhead:

- **Flat data:** Each record is a single line of pipe-separated values. No indentation beyond the initial 4-space field prefix.
- **Nested data:** Child records require additional indentation (4 spaces for one level, 8 for two). At 1,000 records with one nesting level, this adds ∼5,000 tokens.

The indentation overhead is ONTO's primary structural cost. It is the price paid for human-readable hierarchy without brace matching.

6.2 Delimiter Overhead

We measured the token cost of ONTO's delimiters:

- **Pipe (|):** Tokenizes as 1 token in cl100k_base. At 6 fields × 1,000 records = 6,000 pipes = 6,000 tokens.
- **Caret (^):** Also 1 token. Arrays with 3 elements add 2 carets per value.

- **Backticks:** 1 token each. Used only when values contain reserved characters—rare in operational data.

In contrast, JSON requires quotes (2 tokens per string value), colons (1 token per field), braces (2 tokens per record), brackets (2 tokens per array), and commas (1 token per element). The delimiter economy favors ONTO by approximately 3:1 per record.

6.3 Schema Declaration Cost

ONTO's schema declaration—the entity line plus field name lines—is a fixed cost independent of record count. For IoT Telemetry with 8 fields (including nested location), the schema costs approximately 25 tokens:

```
Telemetry[1000]:
    device_id:
    timestamp:
    temperature:
    ...
```

In JSON, these same field names repeat with every record: 8 fields × 1,000 records × ∼3 tokens per key = ∼24,000 tokens for keys alone. The crossover point where ONTO becomes more efficient is approximately 2 records.

6.4 Nesting Depth Impact

Our measured datasets provide two data points on nesting depth:

- **Depth 0 (flat):** 50.8% reduction (System Metrics), 47.3% (Log Entries)
- **Depth 1:** 46.3% reduction (IoT Telemetry)

The 4-percentage-point gap between flat and single-level nesting reflects indentation overhead. The schema-once benefit persists regardless of depth; only the indentation cost increases. Systematic evaluation across deeper nesting levels remains future work.

7 Discussion

7.1 When to Use ONTO

ONTO provides value when:

- Datasets contain 100+ records with repeated structure
- Token costs or context limits are binding constraints
- Data is consumed by LLMs for analytical tasks
- Nesting depth tested up to 2 levels; deeper nesting remains unexplored

ONTO is *not* appropriate for API communication (use JSON), configuration files (use YAML), persistent storage (use Parquet), small datasets where overhead is negligible, or deeply nested structures where indentation costs dominate.

7.2 Integration Pattern

ONTO functions as a preprocessing layer requiring no downstream changes:

```
import onto
data = fetch_metrics()  # List of dicts
onto_str = onto.dumps(data, entity="Metrics")
prompt = f"Analyze:\n{onto_str}\nReport␣anomalies."
response = llm.complete(prompt)
```

The LLM consumes ONTO; the response is natural language. Existing pipelines require only the addition of a serialization call.

7.3 Economic Implications

For workloads where structured datasets are directly embedded in prompts at scale, token reduction translates to proportional cost savings. API pricing scales linearly with input tokens; a 50% reduction halves the input component of inference cost.

7.4 Limitations

Synthetic evaluation only. All benchmarks use synthetic datasets. Validation on production traces with realistic value distributions, edge cases, and schema complexity remains future work.

Single tokenizer. Results use `cl100k_base`. Cross-tokenizer validation (Llama, Mistral, Gemma) would strengthen generalization claims.

No nesting depth ablation. We did not systematically vary nesting depth in the main evaluation. The 4.5pp gap between flat and nested suggests depth matters, but we lack fine-grained measurements.

No minification baseline. Comparison with schema-aware JSON minification (e.g., removing whitespace, shortening keys) would isolate columnar restructuring's contribution versus simple compression.

Homogeneous records required. All records in an entity must share identical schema. Heterogeneous structures require separate entity declarations or fall back to JSON.

No streaming parser. The current implementation loads complete documents into memory, limiting applicability to very large datasets.

8 Related Work

Data serialization formats. JSON [Bray, 2017] remains the dominant interchange format due to simplicity and universal tooling. YAML [Ben-Kiki et al., 2021] improves readability through significant whitespace. Protocol Buffers [Google, 2008] and Apache Avro provide schema-enforced binary encoding for RPC and storage. Apache Parquet [Apache, 2013] enables columnar analytics on large datasets. These formats target storage, transmission, or interchange—not LLM prompt optimization. ONTO occupies a different niche: ephemeral serialization for context-window-constrained consumers.

Prompt compression. LLMLingua [Jiang et al., 2023] compresses prompts by removing low-information tokens guided by perplexity scores from a small language model. LLMLingua-2 [Pan et al., 2024] improves on this with data-distilled compression. These methods are lossy and task-agnostic—they cannot exploit the regularity of structured data. ONTO is complementary: it provides lossless structural compression before any text-level optimization.

Context window optimization. Retrieval-augmented generation [Lewis et al., 2020] reduces context requirements by retrieving relevant passages dynamically. Batch prompting [Cheng et al., 2023] reduces per-query token overhead by processing multiple examples in a single inference call. Structured prompting [Hao et al., 2022] organizes information hierarchically for better comprehension. These techniques address what to include; ONTO addresses how to serialize what is included.

Schema-aware serialization. Schema-aware JSON optimization approaches such as TOON [TOON, 2024] achieve compression through key shortening and structure normalization. Such methods may perform well on flat data but provide limited benefit on deeply nested structures. ONTO's columnar design addresses both cases through structural redesign rather than optimization of the existing format.

Token efficiency research. Recent work has examined tokenization efficiency across languages [Petrov et al., 2023] and proposed sentence-level prompt compression that preserves semantic coherence under token reduction [Liskavets et al., 2024]. These efforts focus on natural language; ONTO addresses structured data specifically.

9 Future Work

Streaming parser. A streaming implementation would enable processing of datasets exceeding memory capacity, with chunk-wise serialization maintaining the schema-once property across boundaries.

Tokenizer sensitivity analysis. Systematic evaluation across tokenizer families (BPE variants, SentencePiece, Unigram) would quantify how reduction ratios vary with vocabulary construction.

Nesting depth analysis. Our evaluation covers depth 0 (flat) and depth 1. Systematic measurement across deeper nesting levels (2, 3+) would characterize how indentation overhead scales and identify the crossover point where ONTO's advantage diminishes.

Schema validation. A schema definition language for ONTO would enable validation, documentation, and tooling support—trading some simplicity for production robustness.

Tool definition serialization. Preliminary experiments suggest ONTO may effectively serialize LLM function-calling tool definitions, where repeated parameter schemas create similar overhead patterns. Systematic evaluation of tool selection accuracy across varying tool counts remains future work.

Production trace validation. Evaluation on real operational datasets—CDN logs, telemetry streams, application metrics—would validate synthetic benchmark findings and reveal edge cases.

Compact hierarchy encoding. Dot notation (e.g., `location.lat` instead of indented blocks) could eliminate indentation overhead at the cost of visual hierarchy, offering a token-optimized mode for deeply nested structures.

10 Conclusion

We presented ONTO, a columnar notation that reduces LLM input token overhead by 46–51% compared to JSON while preserving human readability and nested structure support. The key insight is that serialization for LLM consumption differs fundamentally from serialization for document interchange: ephemeral, single-consumer payloads do not require per-record schema redundancy.

Ablation analysis revealed that key elimination accounts for the majority of savings, with indentation overhead explaining the performance gap between flat and nested data. Token efficiency translates to concrete benefits: reduced API costs, lower inference latency, and increased effective context capacity.

ONTO occupies a specific niche—columnar, hierarchical, human-readable, LLM-optimized—that no existing format addresses. As LLMs transition from conversational assistants to operational intelligence systems consuming large-scale structured data, serialization optimization becomes a practical necessity. We release the specification, reference implementation, and benchmarks as open-source resources to support adoption and further

research.

Reproducibility

Code, specification, benchmark harnesses, synthetic datasets, and the warm prompt used for comprehension evaluation are available at https://github.com/harsh-aranga/onto.

References

Bray, T. (2017). The JavaScript Object Notation (JSON) Data Interchange Format. *RFC 8259*, IETF.

Ben-Kiki, O., Evans, C., and döt Net, I. (2021). YAML Ain't Markup Language Version 1.2. *yaml.org/spec/1.2*.

Google (2008). Protocol Buffers: Developer Guide. https://protobuf.dev/.

Apache Software Foundation (2013). Apache Parquet. https://parquet.apache.org/.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., and Qiu, L. (2023). LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *arXiv:2310.05736*.

Pan, Z., Wu, Q., Jiang, H., et al. (2024). LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. *arXiv:2403.12968*.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

Hao, Y., et al. (2022). Structured Prompting: Scaling In-Context Learning to 1,000 Examples. *arXiv:2212.06713*.

Dziri, N., et al. (2023). Faith and Fate: Limits of Transformers on Compositionality. *NeurIPS 2023*.

Cheng, Z., Kasai, J., and Yu, T. (2023). Batch Prompting: Efficient Inference with Large Language Model APIs. *arXiv:2301.08721*.

Petrov, A., et al. (2023). Language Model Tokenizers Introduce Unfairness Between Languages. *arXiv:2305.15425*.

Liskavets, B., Ushakov, M., Roy, S., Klibanov, M., Etemad, A., and Luke, S. (2024). Prompt Compression with Context-Aware Sentence Encoding for Fast and Improved LLM Inference. *arXiv:2409.01227*.

TOON: Schema-Aware JSON Optimization. https://github.com/toon-format/toon.

Appendix A Warm Prompt for ONTO Comprehension

The following system prompt was provided for ONTO (warm) scenarios:

```
You are analyzing data in ONTO format. ONTO is a
columnar notation where:
- First line declares: EntityName[count]:
- Each field has its own line: fieldname: val1|val2|val3
- Pipe (|) separates values across records
- Indented blocks represent nested structures
- Empty values (||) represent null
- The ^ character separates array elements

Example:
Sensors[2]:
    device_id: sensor-001|sensor-002
    temperature: 23.5|24.1
    status: active|inactive
    location:
        lat: 37.77|37.78
        lon: -122.41|-122.42

Parse data by reading each field line and matching
pipe-separated values to record positions.
```

This prompt adds approximately 200 tokens.